

Dokumen Teknis: Machine Learning Data Iklim

Proyek: Geospatial Data Utilization Platform for Climate Action **Disusun untuk:** Badan Informasi Geospasial (BIG) — Direktorat Atlas dan Pemanfaatan Informasi Geospasial **Tanggal:** 4 Juni 2026 **Versi:** 1.1 **Penyusun:** Tim Climate ML **Referensi hulu:** *Laporan Proof of Concept: ETL Data Iklim v4.1* ([laporan-poc-etl-climate-data.md](#)) — Phase 1-4 (ETL core, RDTR×Climate overlay, QC, CHIRPS, Docker)

Daftar Isi

1. Ringkasan Eksekutif
2. Tujuan & Ruang Lingkup
3. Sumber Data & Kontrak Data (Data Contract)
4. Use Case ML & Pemilihan Model
5. Arsitektur Aplikasi
6. Struktur Folder Proyek
7. Skema Data Internal (Feature Store & Artefak)
8. Feature Engineering
9. Pipeline ML (Train → Evaluate → Predict)
10. Serving (REST API)
11. Strategi Pengujian (Testing)
12. Konfigurasi & Lingkungan
13. Keterbatasan & Rekomendasi
14. Lampiran

1. Ringkasan Eksekutif

Dokumen ini merancang lapisan **Machine Learning (ML)** yang dibangun di atas database spasial hasil PoC ETL **v4.1** ([PostgreSQL 16 + PostGIS 3.6.3](#) , **6 tabel**: 4 POINT + 1 MULTIPOLYGON + 1 log). PoC ini mencakup 5 sumber data (BMKG, ERA5, NASA POWER, **CHIRPS**, **RDTR Yogyakarta**), pipeline **quality control** (flag-and-load, kolom `qc_flag`), dan overlay tata ruang × iklim. Tujuannya membuktikan data tersebut dapat dimanfaatkan untuk tugas prediktif + analitik risiko, serta menyiapkan **scaffold aplikasi** yang reproducible, teruji, dan scalable.

Selaras PoC v4.1: ML mengonsumsi `qc_flag` (hanya melatih baris `qc_flag='OK'` , sesuai pendekatan flag-and-load laporan Bab 16), memakai range QC per-sumber (Bab 16.3), dan mereplikasi analisis *climate risk score* per zona RDTR (Bab 14) sebagai UC-5.

Stack utama: Python 3.12, scikit-learn, pandas/GeoPandas/xarray, SQLAlchemy + GeoAlchemy2 (re-use dari ETL), XGBoost (opsional), FastAPI (serving), MLflow (experiment tracking), Prefect (orquestrasi — konsisten dengan stack ETL), pytest (testing).

Empat use case ML dipetakan langsung ke 3 tabel sumber:

Kode	Use Case	Tabel Sumber	Tipe ML	Target
UC-1	Klasifikasi kondisi cuaca	<code>bmkg_forecast</code>	Klasifikasi multi-kelas	<code>cuaca</code>
UC-2	Prediksi parameter iklim bulanan	<code>nasa_power_monthly</code> , <code>era5_monthly</code>	Regresi / time series	<code>t2m</code> , <code>prectotcorr</code>
UC-3	Deteksi anomali / Quality Control	tabel data + <code>qc_flag</code>	Unsupervised + rule-based	flag <code>is_anomaly</code>
UC-4	Interpolasi spasial suhu (downscaling)	<code>era5_monthly</code>	Regresi spasial	<code>t2m_celsius</code> di koordinat sembarang

Kode	Use Case	Tabel Sumber	Tipe ML	Target
UC-5	Climate risk score per zona RDTR	<code>rdtr_pola_ruang</code> × <code>nasa_power_monthly</code>	Analitik komposit (overlay)	<code>risk_score</code> per zona

Sumber data baru v4.1 yang dipakai/disiapkan: **CHIRPS** (`chirps_monthly` , curah hujan satelit grid Yogyakarta — kandidat fitur/target presipitasi resolusi tinggi) dan **RDTR** (`rdtr_pola_ruang` , 14 zona Kota Yogyakarta — basis UC-5).

Catatan kejujuran data: volume data PoC sangat kecil (mis. NASA POWER hanya 48 baris, 12 bulan × 4 lokasi). Model tidak akan menghasilkan akurasi produksi. **Yang dibuktikan di tahap ini adalah arsitektur, kontrak data, pipeline, dan kerangka evaluasi/uji** — bukan performa final. Setiap use case dirancang agar langsung naik kelas saat data bertambah.

2. Tujuan & Ruang Lingkup

2.1 Tujuan

1. **Memvalidasi kelayakan ML** — membuktikan data PostGIS hasil ETL dapat dikonsumsi untuk training & inference.
2. **Menyediakan scaffold reproducible** — struktur folder, konfigurasi, dan pipeline standar.
3. **Menetapkan kerangka evaluasi & uji** — metrik, quality gate, dan test suite sebelum produksi.
4. **Mengisi gap PoC ETL** — terutama *quality control* (UC-3), yang dicatat sebagai keterbatasan #3 di laporan ETL.

2.2 Ruang Lingkup

Aspek	Dalam Lingkup	Di Luar Lingkup
Use case	UC-1 (utama, end-to-end), UC-2/3/4 (kerangka + stub model)	Deep learning (LSTM/transformer), nowcasting radar
Data	3 tabel PostGIS PoC (2023, Sulsel + Jabar)	Data historis multi-tahun, satelit, CHIRPS
Serving	REST API (FastAPI), batch predict	Real-time streaming, model serving GPU
Orkestrasi	Pipeline training manual + Prefect flow	MLOps penuh (auto-retrain, drift monitor produksi)
Infrastruktur	Lokal / Docker	Kubernetes, model registry terkelola

2.3 Hubungan dengan PoC ETL

ML ini adalah **lapisan di atas tier Hot (PostGIS)** pada arsitektur storage 3-tier dari laporan ETL. ML **membaca** dari tabel `bmkg_forecast` , `era5_monthly` , `nasa_power_monthly` , dan **menulis kembali** hasil prediksi/flag ke tabel baru (`ml_predictions` , `ml_anomalies`) sehingga konsisten dan dapat di-*query* spasial.

Sumber → [ETL: Extract/Transform/Load] → PostGIS (3 tabel) → [ML: Feature → Train → Predict] → PostGIS (tabel ML) → API/SDSS
 ▲ batas PoC ETL ▲ cakupan dokumen ini

3. Sumber Data & Kontrak Data (Data Contract)

ML hanya boleh bergantung pada **kontrak data** yang stabil, bukan detail implementasi ETL. Berikut kolom yang dikonsumsi per tabel (lihat Bab 9 laporan ETL untuk skema penuh).

3.1 bmkf_forecast (UC-1, UC-3)

Kolom	Tipe	Peran ML	Catatan
suhu_c	FLOAT	fitur	°C
kelembaban_pct	FLOAT	fitur	0-100
curah_hujan_mm	FLOAT	fitur	≥ 0
kecepatan_angin_kmh	FLOAT	fitur	≥ 0
arah_angin_deg	FLOAT	fitur	0-360 → di-encode siklik
tutupan_awan_pct	FLOAT	fitur	0-100
cuaca	VARCHAR(100)	target (UC-1)	label kelas (Cerah, Berawan, Hujan Ringan, ...)
datetime_local	VARCHAR(30)	fitur turunan	→ jam, bulan, musim
geom	POINT(4326)	fitur turunan	→ lat, lon

3.2 nasa_power_monthly (UC-2)

Kolom	Tipe	Peran ML
t2m, t2m_max, t2m_min	FLOAT	target/fitur (°C)
allsky_sfc_sw_dwn	FLOAT	fitur (radiasi, MJ/m²/hari)
rh2m	FLOAT	fitur (kelembaban %)
ws2m	FLOAT	fitur (angin m/s)
prctotcorr	FLOAT	target/fitur (curah hujan mm/hari)
year, month	INT	fitur temporal (siklik)
lat, lon, location_label	FLOAT/VARCHAR	fitur spasial

3.3 era5_monthly (UC-2, UC-4)

Kolom	Tipe	Peran ML
t2m_celsius	FLOAT	target
lat, lon	FLOAT	fitur spasial (grid 0.25°)
year, month	INT	fitur temporal
geom	POINT(4326)	fitur spasial / join

3.3b chirps_monthly (UC-2 presipitasi, kandidat fitur)

Curah hujan satelit CHIRPS v3.0, grid 0.05° Kota Yogyakarta (laporan Bab 17).

Kolom	Tipe	Peran ML
precipitation_mm	FLOAT	target/fitur (mm/bulan)
lat, lon	FLOAT	fitur spasial (grid ~5,5 km)
year, month	INT	fitur temporal
qc_flag	VARCHAR	filter QC

3.3c rdtr_pola_ruang (UC-5)

Zona tata ruang RDTR Kota Yogyakarta, 14 zona MULTIPOLYGON (laporan Bab 9/13).

Kolom	Tipe	Peran ML
<code>kategori_zona</code>	VARCHAR	atribut zona (Permukiman/Komersial/RTH)
<code>luas_ha</code>	FLOAT	atribut zona
<code>geom</code> → centroid	MULTIPOLYGON	join nearest-point ke iklim (<code>ST_Centroid</code>)

3.4 Aturan Kontrak (di-enforce lewat validasi & test)

- **Range fisik per-sumber** (selaras QC laporan v4.1 Bab 16.3): BMKG `-10 ≤ suhu_c ≤ 50` ; NASA/ERA5 `-50 ≤ t2m ≤ 60` ; `0 ≤ kelembaban/rh2m ≤ 100` ; CHIRPS `0 ≤ precipitation_mm ≤ 2000` ; `0 ≤ arah_angin_deg ≤ 360` .
- **QC-aware (`qc_flag`)**: training hanya memakai baris `qc_flag = 'OK'` (`filter_qc_clean`), sesuai pendekatan **flag-and-load** laporan (data bermasalah tetap di DB, ditandai, tapi tidak dilatih).
- **Missing value**: NASA POWER memakai sentinel `-999.0` → harus sudah `NULL` /ber- `qc_flag` (tanggung jawab ETL); ML tetap memvalidasi ulang.
- **Koordinat**: `geom` adalah `POINT(lon, lat)` (urutan GIS x,y) — sesuai gotcha laporan. Helper `ST_X` =lon, `ST_Y` =lat; untuk polygon RDTR dipakai `ST_Centroid` .
- **Kardinalitas minimum** untuk training: jika baris < ambang, pipeline **gagal cepat**.

4. Use Case ML & Pemilihan Model

UC-1 — Klasifikasi Kondisi Cuaca (*utama, end-to-end*)

- **Definisi**: prediksi `cuaca` (kategori) dari fitur meteorologi.
- **Manfaat**: gap-filling label cuaca, konsistensi QC, dasar nowcasting saat data observasi tersedia.
- **Model baseline → kandidat**: `DummyClassifier` (baseline) → `RandomForestClassifier` → `XGBoost` / `LightGBM` .
- **Penanganan kelas tak seimbang**: `class_weight="balanced"` , metrik **macro-F1** (bukan akurasi mentah).
- **Validasi**: Stratified K-Fold; karena data temporal, sediakan opsi *time-based split* (latih bulan awal → uji bulan akhir) untuk menghindari kebocoran.
- **Quality gate**: macro-F1 holdout ≥ baseline + margin (lihat Bab 9.4).

UC-2 — Prediksi Parameter Iklim Bulanan

- **Definisi**: regresi `t2m` (atau `prectotcorr`) bulanan dari fitur spasio-temporal + parameter lain.
- **Model**: `GradientBoostingRegressor` / `XGBoost` ; baseline = klimatologi (rata-rata bulanan historis).
- **Roadmap data bertambah**: SARIMA/Prophet (per-lokasi), lalu LSTM/Temporal Fusion Transformer (multi-lokasi, multi-tahun).
- **Metrik**: MAE, RMSE, R²; dibandingkan terhadap baseline klimatologi (skill score).

UC-3 — Deteksi Anomali / Quality Control

- **Definisi**: menandai pengukuran tak wajar (sensor error, outlier spasial/temporal).
- **Pendekatan dua lapis**: 1. **Rule-based** (deterministik): range check fisik (Bab 3.4) → cepat, dapat dijelaskan. 2. **Unsupervised** (`IsolationForest`): menangkap kombinasi nilai janggal yang lolos range check.
- **Output**: kolom `is_anomaly` + `anomaly_score` + `reason` , ditulis ke tabel `ml_anomalies` .
- **Manfaat**: mengisi keterbatasan QC laporan ETL (rekomendasi prioritas tinggi #2).

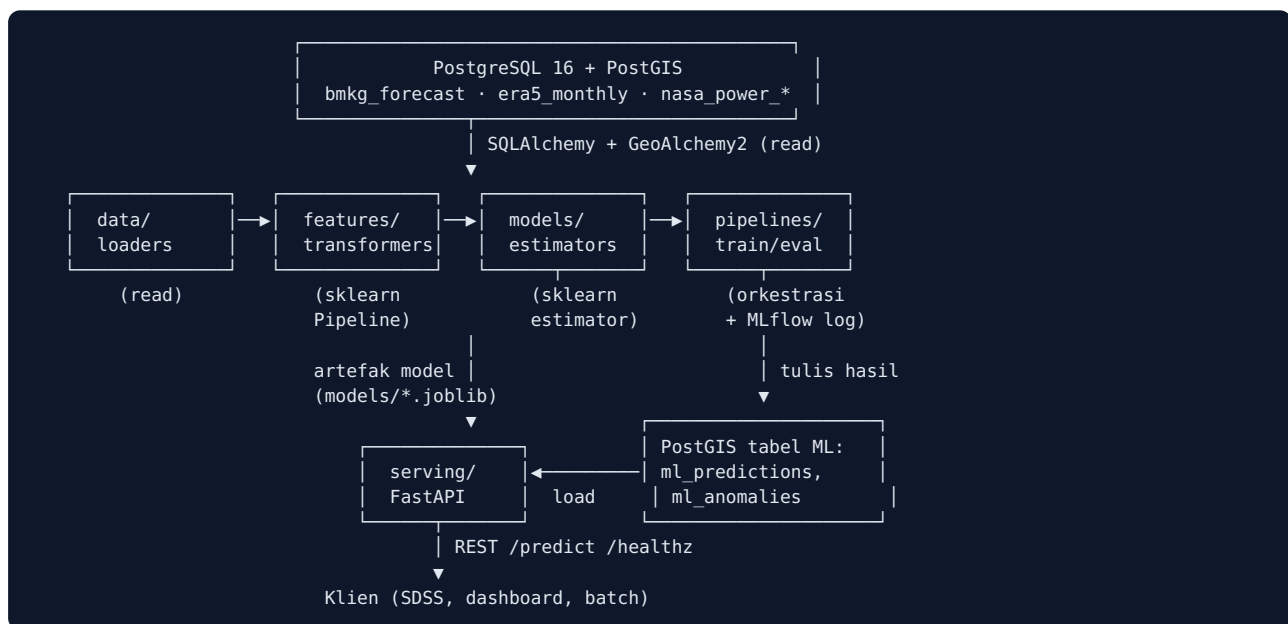
UC-4 — Interpolasi Spasial Suhu (Downscaling)

- **Definisi**: prediksi `t2m_celsius` pada koordinat sembarang dari grid ERA5.
- **Model**: baseline IDW/nearest; kandidat `RandomForestRegressor` (lat, lon, month) → roadmap Kriging (`scikit-gstat` / `pykrige`).
- **Manfaat**: menghasilkan permukaan suhu kontinu untuk lokasi tanpa grid (mis. titik desa BMKG).

UC-5 — Climate Risk Score per Zona RDTR (overlay tata ruang × iklim)

- **Definisi:** mereplikasi analisis overlay Phase 3 (laporan Bab 14) — setiap zona RDTR Kota Yogyakarta diberi data iklim dari titik NASA POWER terdekat (*nearest point* ke centroid), lalu dihitung **composite risk score** ternormalisasi antar-zona: $\text{risk} = 0.5 \cdot \text{norm}(\text{suhu_maks}) + 0.3 \cdot \text{norm}(\text{curah_hujan}) + 0.2 \cdot \text{norm}(\text{kelembaban})$.
- **Sifat:** analitik deterministik (bobot dari domain), bukan model terlatih — disajikan sebagai endpoint `/v1/risk/zones` + panel frontend (tabel ranking + level Rendah/Sedang/Tinggi).
- **Roadmap ML:** saat tersedia label dampak nyata (banjir/kekeringan historis per zona), bobot dapat dipelajari (regresi/klasifikasi tersupervisi) menggantikan bobot tetap.
- **Modul:** `models/climate_risk.py`.

5. Arsitektur Aplikasi



Prinsip desain: - **Pemisahan tegas (separation of concerns):** `data` (I/O) ⊥ `features` (transformasi) ⊥ `models` (estimator) ⊥ `pipelines` (orkestrasi) ⊥ `serving` (API). Tiap lapisan diuji terpisah. - **Semua transformasi** → `sklearn.Pipeline` agar identik antara training & serving (mencegah training-serving skew). - **Konfigurasi via file** (`config/*.yaml`) + `env var` (`.env`) — tidak ada nilai hard-coded. - **Artefak versioned** — model disimpan dengan metadata (versi data, hash fitur, metrik) lewat MLflow + `joblib`.

6. Struktur Folder Proyek

```
climate-ml/
├── README.md                # Cara setup, train, test, serve
├── pyproject.toml           # Metadata paket + konfigurasi tool (ruff, pytest)
├── requirements.txt         # Dependencies (pin minor version)
├── Makefile                # Shortcut: make install/train/test/serve
├── .env.example            # Template kredensial DB (jangan commit .env)
├── .gitignore
├── config/
│   ├── config.yaml         # Konfigurasi default (DB, path, seed)
│   └── models/
│       ├── uc1_weather_clf.yaml # Hyperparameter + fitur UC-1
│       └── uc2_climate_reg.yaml  # Hyperparameter + fitur UC-2
├── docs/
│   └── dokumen-teknis-ml-climate.md # Dokumen ini
└── sql/
```

```

└─ 01_ml_tables.sql          # DDL tabel ml_predictions & ml_anomalies (PostGIS)

src/
└─ climate_ml/
   ├── __init__.py
   ├── config.py             # Loader config (pydantic-settings)
   ├── data/
   │   ├── __init__.py
   │   ├── db.py             # Engine/session SQLAlchemy ke PostGIS
   │   ├── loaders.py        # load_bmkg(), load_nasa_power(), load_era5()
   │   └── validation.py     # Validasi kontrak data (range, null, kardinalitas)
   ├── features/
   │   ├── __init__.py
   │   ├── temporal.py       # encoding siklik jam/bulan, fitur musim
   │   ├── spatial.py        # ekstraksi lat/lon dari geom, fitur geografis
   │   └── build.py          # ColumnTransformer per use case
   ├── models/
   │   ├── __init__.py
   │   ├── registry.py       # factory: nama → estimator
   │   ├── weather_classifier.py # UC-1
   │   ├── climate_regressor.py # UC-2
   │   ├── anomaly_detector.py # UC-3 (rule + IsolationForest)
   │   ├── spatial_interpolator.py # UC-4
   │   └── climate_risk.py   # UC-5 (overlay risk score RDTR×iklim)
   ├── pipelines/
   │   ├── __init__.py
   │   ├── train.py          # CLI: latih + log MLflow + simpan artefak
   │   ├── evaluate.py       # metrik + quality gate
   │   ├── predict.py        # batch predict → tulis ke PostGIS
   │   └── flow.py           # Prefect flow (opsional, orkestrasi)
   ├── serving/
   │   ├── __init__.py
   │   ├── api.py            # FastAPI app
   │   └── schemas.py        # Pydantic request/response
   └── utils/
       ├── __init__.py
       ├── logging.py
       ├── metrics.py        # macro-F1, MAE/RMSE, skill score
       └── io.py             # save/load artefak + metadata

models/
└─ .gitkeep                  # Artefak terlatih (.joblib) – di-gitignore

notebooks/
└─ 01_eda.ipynb             # Eksplorasi (EDA, prototyping)

data/
└─ fixtures/                # Cache lokal/fixture (di-gitignore kecuali fixtures)
   ├── bmkg_sample.csv      # Sampel CSV untuk test (tanpa perlu DB)
   └─ nasa_power_sample.csv

tests/
└─ __init__.py
   ├── conftest.py          # Fixture bersama (data sintetis, config test)
   ├── unit/
   │   ├── test_validation.py
   │   ├── test_features.py
   │   ├── test_metrics.py
   │   └── test_registry.py
   ├── integration/
   │   ├── test_train_pipeline.py # train end-to-end pada fixture
   │   └── test_predict_pipeline.py
   ├── api/
   │   └── test_api.py       # TestClient FastAPI
   └─ data/
       └── test_data_contract.py # validasi schema & range pada data nyata (opsional, butuh DB)

```

7. Skema Data Internal (Feature Store & Artefak)

7.1 Tabel hasil ML (ditulis kembali ke PostGIS)

sql/01_ml_tables.sql :

```
-- Hasil prediksi (UC-1, UC-2, UC-4)
CREATE TABLE IF NOT EXISTS ml_predictions (
  id          SERIAL PRIMARY KEY,
  use_case    VARCHAR(20) NOT NULL, -- 'UC1' | 'UC2' | 'UC4'
  model_name  VARCHAR(100) NOT NULL,
  model_version VARCHAR(50) NOT NULL,
  target      VARCHAR(50) NOT NULL, -- 'cuaca' | 't2m' | ...
  predicted   VARCHAR(100) NOT NULL, -- nilai/label prediksi (string agar generik)
  proba       FLOAT,               -- confidence (klasifikasi) / interval (regresi)
  ref_year    INTEGER,
  ref_month   INTEGER,
  geom        geometry(Point, 4326),
  predicted_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (use_case, model_version, target, ref_year, ref_month, geom)
);
CREATE INDEX IF NOT EXISTS idx_ml_pred_geom ON ml_predictions USING GIST (geom);

-- Hasil deteksi anomali (UC-3)
CREATE TABLE IF NOT EXISTS ml_anomalies (
  id          SERIAL PRIMARY KEY,
  source_table VARCHAR(50) NOT NULL, -- 'bmkg_forecast' | ...
  source_id   INTEGER NOT NULL,
  is_anomaly  BOOLEAN NOT NULL,
  anomaly_score FLOAT,
  reason      TEXT,               -- 'suhu>60' | 'isolation_forest' | ...
  geom        geometry(Point, 4326),
  detected_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_ml_anom_geom ON ml_anomalies USING GIST (geom);
```

Konsisten dengan konvensi PoC ETL: kolom `geom` POINT/4326 + GiST index + kolom timestamp.

7.2 Artefak model

Setiap model disimpan sebagai `models/<use_case>_<name>_<version>.joblib` berisi: `sklearn.Pipeline` lengkap (preprocessing + estimator) + sidecar `*.json` metadata:

```
{
  "use_case": "UC1",
  "model_name": "random_forest",
  "model_version": "2026.06.04-1",
  "trained_at": "2026-06-04T10:00:00Z",
  "data_rows": 138,
  "feature_names": ["suhu_c", "kelembaban_pct", "..."],
  "target": "cuaca",
  "metrics": {"macro_f1": 0.71, "accuracy": 0.78},
  "baseline_metrics": {"macro_f1": 0.18},
  "git_sha": "abc1234",
  "config_hash": "...",
}
```

8. Feature Engineering

Semua transformasi dibungkus `sklearn.compose.ColumnTransformer` di dalam `sklearn.Pipeline`.

8.1 Fitur temporal (`features/temporal.py`)

- **Encoding siklik** untuk hindari diskontinuitas (Desember↔Januari, 23:00↔00:00): `month_sin = sin(2π·month/12)` , `month_cos = cos(2π·month/12)` ; analog untuk `hour` .
- **Musim** (kategori): basah (Nov–Apr) / kemarau (Mei–Okt) — relevan iklim tropis Indonesia (lihat pola laporan ETL Bab 8.3).

8.2 Fitur spasial (`features/spatial.py`)

- `lat = ST_Y(geom)` , `lon = ST_X(geom)` (lihat gotcha urutan koordinat).
- Proxy ketinggian/karakteristik: pesisir vs dataran tinggi (dari `location_label` /lat-lon) — selaras observasi laporan ETL (Toraja/Bandung lebih dingin).

8.3 Fitur meteorologi (UC-1)

- `arah_angin_deg` → siklik (`sin` / `cos`).
- Numerik (`suhu_c` , `kelembaban_pct` , ...) → `StandardScaler` (penting untuk model jarak/linear; pohon tidak butuh tapi tetap aman).
- Imputasi: `SimpleImputer(strategy="median")` sebagai jaring pengaman.

8.4 Pencegahan kebocoran (leakage)

- `curah_hujan_mm` **tidak** dipakai sebagai fitur UC-1 jika kelas `cuaca` ditentukan oleh hujan (target leakage) — diatur eksplisit lewat daftar fitur di `config/models/uc1_weather_clf.yaml` .
- Scaler & imputer **fit hanya pada train fold** (otomatis karena di dalam Pipeline + CV).

9. Pipeline ML

9.1 Train (`pipelines/train.py`)

Langkah: load → validasi kontrak → build Pipeline (preprocess+estimator) → CV → fit pada train → evaluasi holdout → quality gate → log MLflow → simpan artefak.

```
python -m climate_ml.pipelines.train --use-case UC1 --config config/models/uc1_weather_clf.yaml
```

9.2 Evaluate (`pipelines/evaluate.py`)

Hitung metrik pada holdout + bandingkan baseline; cetak laporan klasifikasi/regresi.

9.3 Predict (`pipelines/predict.py`)

Muat artefak → inference batch dari PostGIS → tulis ke `ml_predictions` / `ml_anomalies` (idempoten lewat UNIQUE constraint).

```
python -m climate_ml.pipelines.predict --use-case UC1 --model models/UC1_random_forest_latest.joblib
```

9.4 Quality Gate

Build/deploy **ditolak** bila model tidak melampaui baseline:

Use case	Metrik	Ambang minimum
UC-1	macro-F1 (holdout)	$\geq$ <code>DummyClassifier(most_frequent)</code> macro-F1 + 0.10
UC-2	skill score vs klimatologi	> 0 (MAE model < MAE baseline)
UC-3	recall rule-based pada anomali tertanam	$= 1.0$ (tidak boleh lolos range check)

Quality gate diuji otomatis (lihat `tests/integration/test_train_pipeline.py`).

10. Serving (REST API)

FastAPI (`serving/api.py`), validasi I/O dengan Pydantic (`serving/schemas.py`).

Endpoint	Method	Fungsi
<code>/healthz</code>	GET	Liveness + model termuat?
<code>/v1/predict/weather</code>	POST	UC-1: fitur meteorologi → kelas cuaca + proba
<code>/v1/predict/climate</code>	POST	UC-2: lokasi+bulan → prediksi t2m/curah hujan
<code>/v1/anomaly/check</code>	POST	UC-3: cek satu pengukuran → <code>is_anomaly</code> + reason

Contoh request `/v1/predict/weather`:

```
{ "suhu_c": 28.0, "kelembaban_pct": 87, "kecepatan_angin_kmh": 5.1,
  "arah_angin_deg": 146, "tutupan_awan_pct": 100, "lat": -5.16, "lon": 119.40,
  "datetime_local": "2026-05-20T20:00:00" }
```

Response:

```
{ "predicted": "Berawan", "proba": 0.62, "model_version": "2026.06.04-1" }
```

Menjalankan: `uvicorn climate_ml.serving.api:app --reload`.

11. Strategi Pengujian (Testing)

Tiga tingkat, semua dijalankan dengan **pytest**. Test default **tidak butuh database** — memakai fixture/data sintesis — agar bisa jalan di CI mana pun. Test yang butuh DB ditandai marker `@pytest.mark.db` dan di-skip otomatis bila env DB tak tersedia.

11.1 Piramida Test



- api/ – FastAPI TestClient (kontrak endpoint)
- integration – pipeline train/predict end-to-end pada fixture
- unit – validasi, fitur, metrik, registry (cepat, mayoritas)

11.2 Cakupan per modul

Test	Memverifikasi
<code>unit/test_validation.py</code>	range check menolak suhu 999, kelembaban 150; lolos data wajar; gagal-cepat saat baris < ambang
<code>unit/test_features.py</code>	encoding siklik benar (month 12 ≈ month 0); lat/lon dari geom POINT(lon,lat) tidak tertukar
<code>unit/test_metrics.py</code>	macro-F1, MAE/RMSE, skill score sesuai nilai acuan manual
<code>unit/test_registry.py</code>	factory mengembalikan estimator yang benar; nama tak dikenal → error jelas
<code>integration/test_train_pipeline.py</code>	training pada fixture menghasilkan artefak; quality gate lewat ; Pipeline reusable untuk predict
<code>integration/test_predict_pipeline.py</code>	predict menghasilkan kolom & tipe sesuai skema <code>ml_predictions</code>
<code>api/test_api.py</code>	

Test	Memverifikasi
	<code>/healthz 200</code> ; <code>/v1/predict/weather</code> mengembalikan label valid; input invalid → 422
<code>data/test_data_contract.py</code> (marker <code>db</code>)	data nyata di PostGIS memenuhi kontrak Bab 3.4

11.3 Quality test khusus ML

- **Determinisme:** training dengan `random_state` tetap → metrik identik antar run (uji reproducibility).
- **Invarians:** menggeser semua suhu +0°C tidak mengubah prediksi; baris duplikat tidak crash.
- **Tidak ada training-serving skew:** Pipeline yang sama dipakai di train & API (diuji dengan memuat artefak lalu memanggil endpoint).
- **Anti-leakage:** assert daftar fitur UC-1 tidak memuat kolom terlarang.

11.4 Cara menjalankan

```
make test           # semua test (skip yang butuh DB jika DB absen)
pytest tests/unit -v # hanya unit (cepat, <5 detik)
pytest -m "not db"  # eksplisit tanpa DB
pytest -m db        # hanya test yang butuh PostGIS (set env DB dulu)
pytest --cov=climate_ml # dengan laporan coverage (target ≥ 80% pada src non-IO)
```

CI (GitHub Actions, opsional): jalankan `ruff check` + `pytest -m "not db" --cov` pada tiap push.

12. Konfigurasi & Lingkungan

- **Python 3.12** (selaras PoC ETL), virtual environment terisolasi.
- **Koneksi DB** dari `.env` (re-use kredensial PoC): `DATABASE_URL=postgresql+psycopg2://climate_user:climate_poc_2026@localhost:5432/climate_etl`
- **Reproducibility:** `RANDOM_SEED=42` global; versi library di-pin di `requirements.txt`.
- **MLflow** lokal: `mlruns/` (file backend) — cukup untuk PoC; produksi → MLflow server.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # lalu isi DATABASE_URL
psql "$DATABASE_URL" -f sql/01_ml_tables.sql # buat tabel ML
make test           # verifikasi scaffold
make train UC=UC1   # latih model UC-1
make serve          # jalankan API
```

13. Keterbatasan & Rekomendasi

No	Keterbatasan	Dampak	Mitigasi
1	Data sangat kecil (PoC)	Model tidak representatif	Skala data dulu (multi-tahun, multi-provinsi) sebelum klaim akurasi
2	Hanya 1 tahun (2023)	Tidak bisa tangkap variasi antar-tahun/tren	Tambah data historis ERA5 (1940-) & NASA POWER (1981-)
3	BMKG = prakiraan, bukan observasi	Label <code>cuaca</code> adalah prakiraan model BMKG	Koordinasi akses data observasi BMKG (selaras rekomendasi ETL 12.2)
4	Belum ada drift monitoring produksi	Model bisa usang diam-diam	Tambah monitoring (Evidently/MLflow) di fase produksi
5	Interpolasi spasial sederhana	Akurasi UC-4 terbatas	Adopsi Kriging + kovariat ketinggian (DEM)

Roadmap (prioritas): 1. **Tinggi:** perbanyak data (multi-tahun/provinsi); aktifkan UC-3 sebagai QC otomatis pada pipeline ETL. 2. **Sedang:** integrasi pipeline training ke Prefect (re-use orchestrator ETL); MLflow server; Dockerfile. 3. **Rendah:** model deret waktu lanjutan (Prophet/LSTM), Kriging untuk UC-4, indikator iklim turunan (SPI, anomali, tren).

14. Lampiran

Lampiran A — Dependencies (requirements.txt)

```
pandas, numpy, scikit-learn, scipy          # core ML
geopandas, shapely                           # spasial
SQLAlchemy, GeoAlchemy2, psycopg2-binary    # baca PostGIS (re-use ETL)
xarray, netcdf4                              # ERA5 (opsional)
xgboost                                       # model boosting (opsional)
fastapi, uvicorn, pydantic, pydantic-settings# serving + config
mlflow                                       # experiment tracking
prefect                                     # orkestrasi (re-use ETL)
joblib, pyyaml                              # artefak + config
pytest, pytest-cov, httpx                  # testing
ruff                                         # linting
```

Lampiran B — Pemetaan Use Case → Test → Quality Gate

Use case	Modul model	Test integrasi	Quality gate
UC-1	weather_classifier.py	test_train_pipeline.py	macro-F1 > baseline + 0.10
UC-2	climate_regressor.py	test_train_pipeline.py	skill score > 0
UC-3	anomaly_detector.py	test_predict_pipeline.py	recall rule-based = 1.0
UC-4	spatial_interpolator.py	(roadmap)	MAE < IDW baseline

Dokumen ini adalah bagian dari proyek Geospatial Data Utilization Platform for Climate Action (2026-2028). Dibangun di atas Laporan PoC ETL Data Iklim v2.1.